

Week4

Creating Microservices for account and loan

In this hands on exercises, we will create two microservices for a bank. One microservice for handling accounts and one for handling loans.

Each microservice will be a specific independent Spring RESTful Webservice maven project having its own pom.xml. The only difference is that, instead of having both account and loan as a single application, it is split into two different applications. These webservices will be a simple service without any backend connectivity.

Follow steps below to implement the two microservices:

Account Microservice

❏ Create folder with employee id in D: drive

❏ Create folder named 'microservices' in the new folder created in previous step. This folder will contain all the sample projects that we will create for learning microservices.

❏ Open <https://start.spring.io/> in browser

❏ Enter form field values as specified below:

o Group: com.cognizant

o Artifact: account

❏ Select the following modules

o Developer Tools > Spring Boot DevTools

o Web > Spring Web

❏ Click generate and download the zip file

❏ Extract 'account' folder from the zip and place this folder in the 'microservices' folder created earlier

❏ Open command prompt in account folder and build using mvn clean package command

❏ Import this project in Eclipse and implement a controller method for getting account details based on account number. Refer specification below:

o Method: GET

o Endpoint: /accounts/{number}

o Sample Response. Just a dummy response without any backend connectivity.

{ number: "00987987973432", type: "savings", balance: 234343 }

❏ Launch by running the application class and test the service in browser

Loan Microservice

❏ Follow similar steps specified for Account Microservice and implement a service API to get loan account details

o Method: GET

o Endpoint: /loans/{number}

o Sample Response. Just a dummy response without any backend

connectivity.

```
{ number: "H00987987972342", type: "car", loan: 400000, emi: 3258, tenure: 18 }
```

☐ Launching this application by having account service already running

☐ This launch will fail with error that the bind address is already in use

☐ The reason is that each one of the service is launched with default port

number as 8080. Account service is already using this port and it is not

available for loan service.

☐ Include "server.port" property with value 8081 and try launching the

application

☐ Test the service with 8081 port

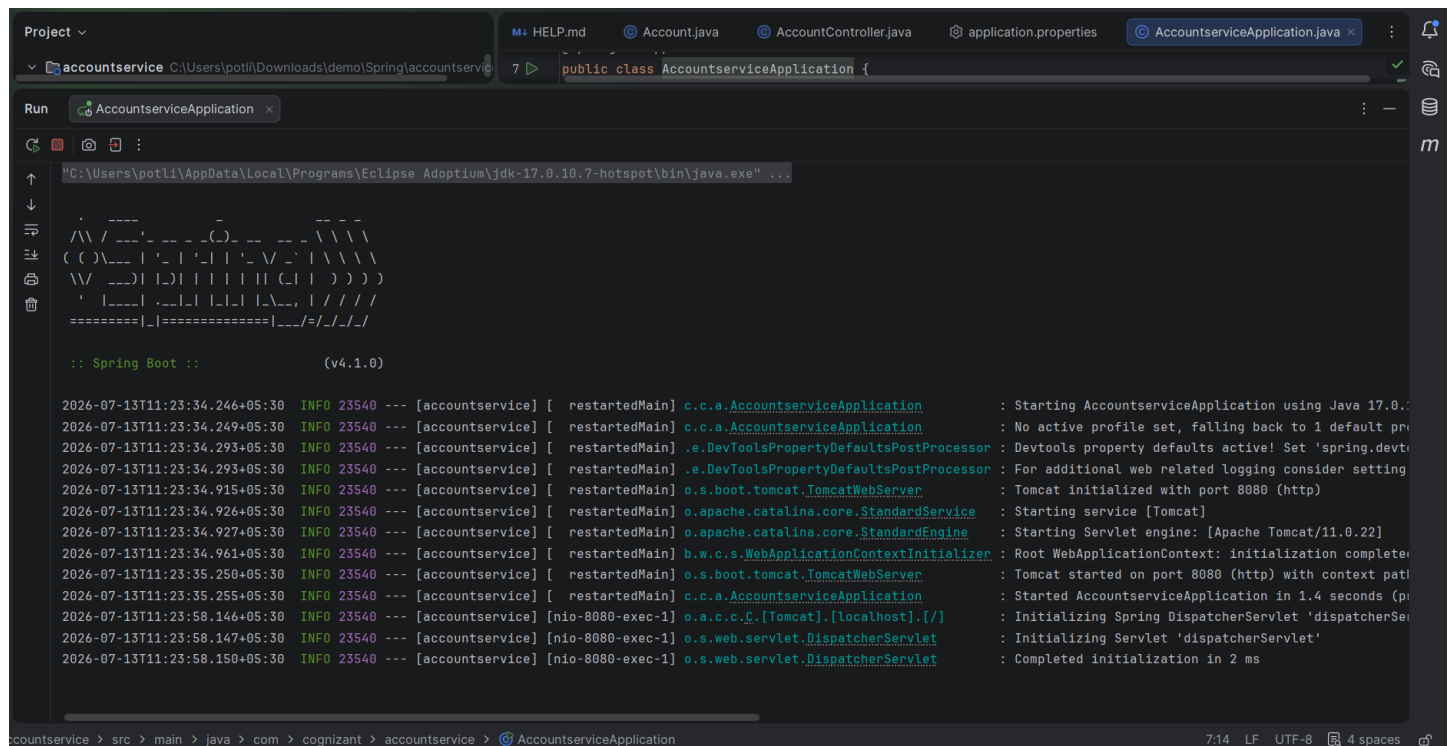
Now we have two microservices running on different ports.

NOTE: The console window of Eclipse will have both the service console

running. To switch between different consoles use the monitor icon within the

console view.

Creating Microservices for account

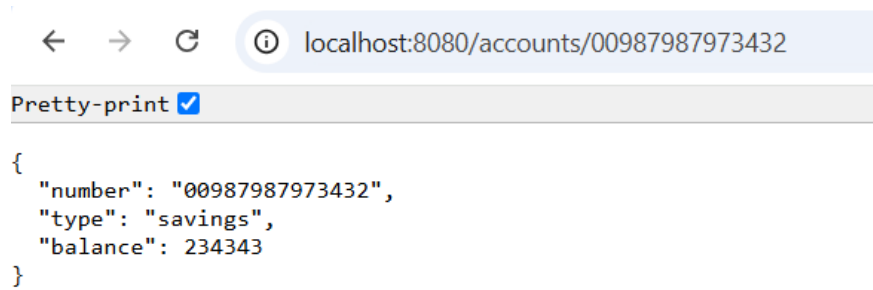


The screenshot shows the Eclipse IDE interface. The top bar displays the project name 'accountservice' and the file path 'C:\Users\potli\Downloads\demo\Spring\accountservice'. The editor window shows the 'AccountServiceApplication.java' file with the following code:

```
public class AccountServiceApplication {
```

The Run console shows the output of the application. The output is as follows:

```
2026-07-13T11:23:34.246+05:30 INFO 23540 --- [accountservice] [ restartedMain] c.c.a.AccountServiceApplication : Starting AccountServiceApplication using Java 17.0.1
2026-07-13T11:23:34.249+05:30 INFO 23540 --- [accountservice] [ restartedMain] c.c.a.AccountServiceApplication : No active profile set, falling back to 1 default profile
2026-07-13T11:23:34.293+05:30 INFO 23540 --- [accountservice] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults active! Set 'spring.devtools.restart.enabled=true' for additional web related logging consider setting
2026-07-13T11:23:34.915+05:30 INFO 23540 --- [accountservice] [ restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-07-13T11:23:34.926+05:30 INFO 23540 --- [accountservice] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-13T11:23:34.927+05:30 INFO 23540 --- [accountservice] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22]
2026-07-13T11:23:34.961+05:30 INFO 23540 --- [accountservice] [ restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed
2026-07-13T11:23:35.250+05:30 INFO 23540 --- [accountservice] [ restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-07-13T11:23:35.255+05:30 INFO 23540 --- [accountservice] [ restartedMain] c.c.a.AccountServiceApplication : Started AccountServiceApplication in 1.4 seconds (process running for 1.4)
2026-07-13T11:23:58.146+05:30 INFO 23540 --- [accountservice] [nio-8080-exec-1] o.a.c.c.g.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-07-13T11:23:58.147+05:30 INFO 23540 --- [accountservice] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-07-13T11:23:58.150+05:30 INFO 23540 --- [accountservice] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
```

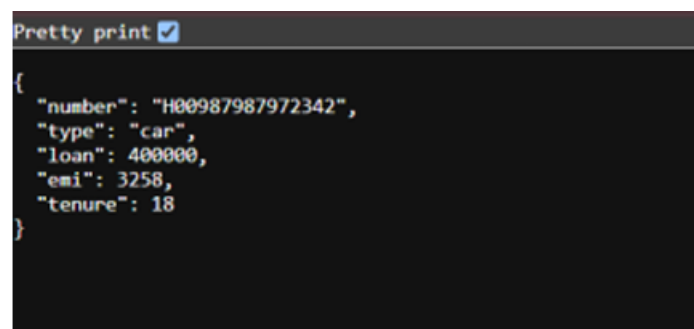


```
localhost:8080/accounts/00987987973432

Pretty-print ☒

{
  "number": "00987987973432",
  "type": "savings",
  "balance": 234343
}
```

Creating Microservices for loan



```
localhost:8080/loans/00987987972342

Pretty print ☒

{
  "number": "H00987987972342",
  "type": "car",
  "loan": 400000,
  "emi": 3258,
  "tenure": 18
}
```